MDC AI-Enabled Messaging SDLC

总体技术方案（修订版）

确定性流程编排 · 阶段级 AI 能力 · 局部返工闭环 · 人工审批治理

Version 2.0 | 2026

文档说明

项目	内容
文档目的	将完整 Messaging SDLC 从“主 Agent 调度多 Agent”修订为“确定性工作流 + 阶段级 AI 能力 + 局部返工 Loop”的企业级技术方案。
适用范围	Use Case Intake、人工审核、Content、合规、Template、UAT Testing、AC Verification、Release Check。
核心原则	AI 不控制批和段合法性； Workflow Orchestrator 控制流程，AI 嵌入各阶段执行适合 AI 的理解、生成、解释和验证任。
Phase 1 关系	现有 Requirement Intake Agent 作为 Stage 1 的阶段级 Agent 保留，不推前改造。

目录

1. 执行摘要
2. 业务背景与问题定义
3. 架构修订结论与设计原则
4. 端到端业务流程
5. 总体逻辑架构
6. 核心组件详细设计
7. 各阶段技术设计与局部 Loop
8. Workflow 状态机、事件与 Gate
9. Issue Routing 与 Impact Analysis
10. Agent、Skill 与 Reducer 运行模型
11. Knowledge 与 Prompt/Skill 治理
12. Case、Artifact、Issue 与 Evidence 数据模型
13. Traceability Graph 与版本失效
14. API、事件与接口契约
15. 安全、限、人工治理
16. 可观测性、评测与 Feedback Improvement Loop
17. 部署与集成方案
18. 分阶段实施路线
19. Phase 1 现状映射与后续影响
20. 验收标准、非功能要求与风险

1. 执行摘要

本方案将 MDC 的完整业务流程定义为一个由确定性 Workflow Orchestrator 驱动的 Messaging SDLC。AI 不再以“大 AI Case Manager”身份控制所有子 Agent，而是作为阶段级能力嵌入 Requirement、Content 与 AC Verification 等高价值环节。每个阶段都有明确输入、输出、检查 Gate 和返工 Loop；审核或验证失败后，由 Review Feedback Processing、Issue Router 与 Impact Engine 将问题精准返回到任段，重做受影的最小范。

- 主流程由状态机、审批 Gate 和规则引擎控制，保证流程可预测、可审计。
- 仅对需要多轮推理、动态决策和自然语言理解的环节使用 Agent。
- 人工审核继续保留最终责任，AI 负责准备、定位、解释、修复与证据组织。
- 跨阶段共享 Skills、Knowledge、Issue Routing、Impact Analysis 与 Traceability。
- Feedback 进入离线改进闭环，用于持续优化 Hint、规则、Prompt、Validator 与 Skill 版本。

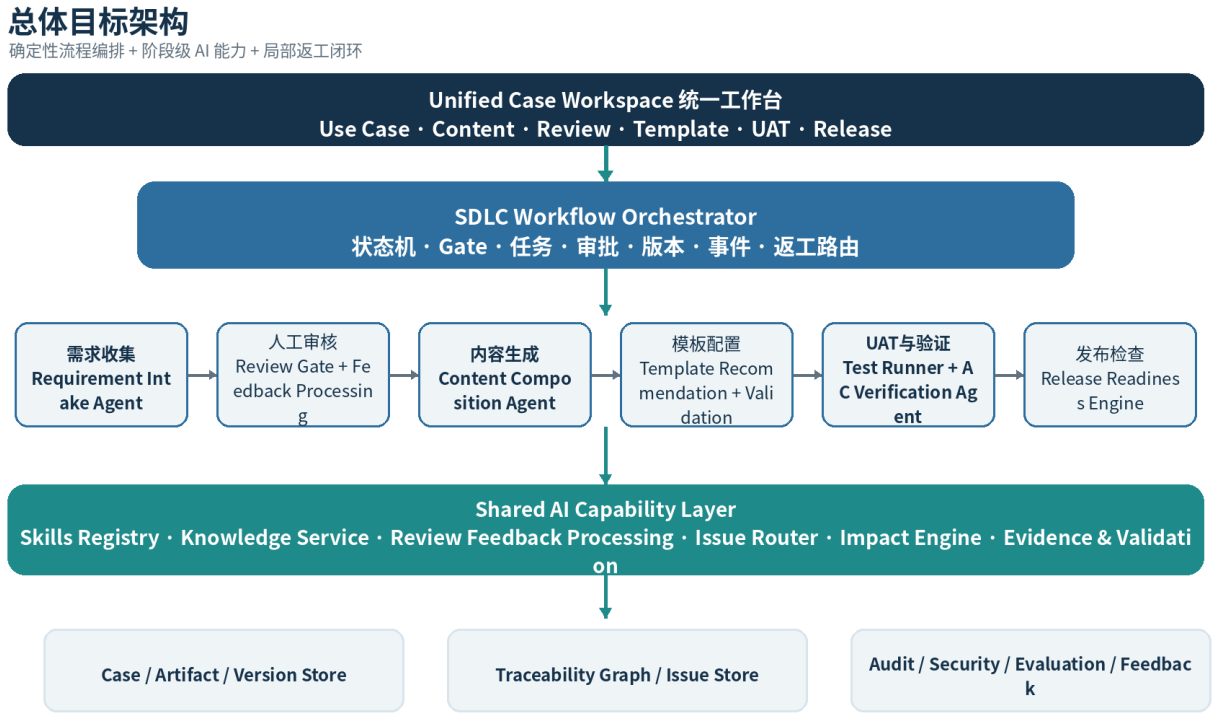


图 1 总体目标架构

2. 业务背景与问题定义

2.1 现行业务流程

业务部门提出消息发送需求后，需要完成 Use Case、人工审核、Content 生成与合规审核、Template 配置、UAT 自动测试、AC Verification、Release Check 及最布。流程具有的序、固定的点和多次返工。

2.2 当前主要问题

问题	业务表现	技术根因
业务字段复杂	用户不了解 33 个字段及 MDC 能力，需要反复询问团队人员	知识、规则与用户引导未系统化
人工参与过多	人工引导填表、编写内容、分析审核意	缺少阶段级 AI 自动化与结构化 Artifact

	见、判断测试结果	
返工范围过大	审核或验证失败后依赖人工判断应返回哪里，常重复修改无关内容	缺少 Issue 分类、跨阶段 Traceability 与 Impact Analysis
缺少反馈闭环	相同持出，核无法淀到下一次流程	缺少 Feedback Store、Skill 版本与离线评测机制
状态与版本割裂	不同模块可能依赖聊天记录或临时数据	缺少统一 Case、Artifact、Version 与 Event Model

2.3 目标业务价值

- 减少 MDC/IT 人员在字段解释、反复沟通、模板修改和测试核对上的重复劳动。
- 提升一次审核通过率和一次测试通过率，减少不必要的整链路返工。
- 将历史成功案例、审核经验和测试证据沉淀为可复用能力。
- 让业务用户始终知道当前阶段、阻塞原因、需要完成的最小动作。
- 建立可审计、可回溯、可逐阶段上线的企业级 AI 体系。

3. 架构修订结论与设计原则

3.1 核心架构结论

总体架构不采用“AI Case Manager 作为主 Agent 自由调度所有 Agent”。主流程本身高度确定，最合适的模式是：Workflow Orchestrator 负责控制流程；阶段级 Agent 负责复杂任务；Review Feedback Processing 与 Issue Router 负责失败后的精准返工；轻量 Case Assistant 仅提供统一用户界面和状态解

。

组件	是否为 Agent	职责
SDLC Workflow Orchestrator	否	控制 Stage、Gate、任务、审批、版本、事件和状态转换
Case Assistant	否/轻量 AI	展示当前阶段、阻塞原因和建议动作，不拥有流程控制权
Requirement Intake Agent	是	多轮引导、知识解释、预填、冲突处理和需求确认
Content Composition Agent	是	基于 Requirement/AC/历史模板生成内容并支持局部修改
AC Verification Agent	是	理解 AC、截图和日志，输出证据化 Verdict
Review Feedback Processing	否，Skills/Service	解析人工审核意见并生成结构化 Issue
Issue Router / Impact Engine	否，规则+模型	判断问题返回阶段及受影响 Artifact
Template Recommendation / Validator	否，Skill+规则	推荐模板并检查配置兼容性
Test Runner	否，确定性服务	执行 UAT 发送、日志收集和截图取证
Release Readiness Engine	否，规则服务	检查所有 Gate、Artifact 和 Blocker

3.2 设计原则

- Workflow first：阶段转换和审批合法性由代码控制。
- Agent only where needed：只有多轮、动态、模糊推理任务才使用 Agent。

- Local remediation：失败后返回责任阶段，只修复受影响范围。
- Artifact first：跨阶段通过结构化 Artifact 协作，不解析长聊天历史。
- Human governed：IT、Compliance 与 Release 的最终审批由人承担。
- Evidence grounded：知识回答、审核解释和 AC Verdict 必须附来源或证据。
- Reducer-only state mutation：Agent Path 的业务状态更新统一经过 Reducer。
- Version everything：Requirement、Content、Template、AC、Test、Evidence 均版本化。

4. 端到端业务流程

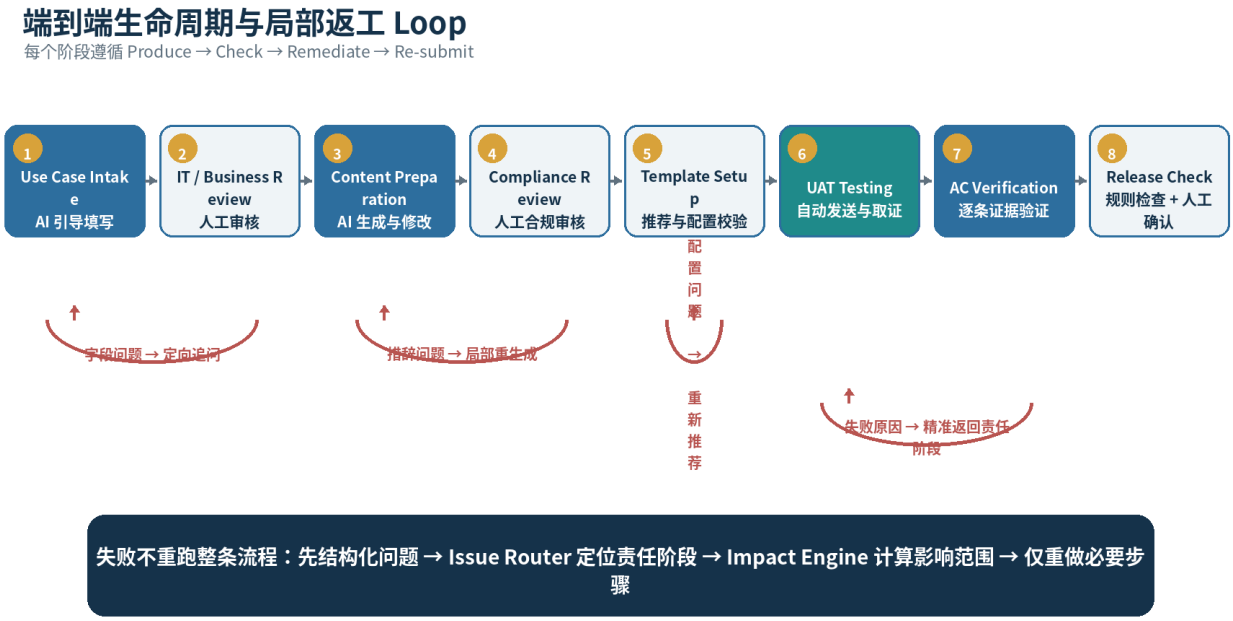


图 2 端到端生命周期与局部返工 Loop

4.1 标准主流程

1. Requirement Intake Agent 引导用户完成 Use Case。
2. IT/Business Reviewer 对 Use Case 进行人工审核。
3. Content Composition Agent 基于 Confirmed Requirement 与 AC 生成内容。
4. Compliance Reviewer 对 Content 进行人工审核。
5. 系统推荐并校验 Template 配置，用户确认。
6. Test Runner 在 UAT 执行多渠道模拟发送并采集 Screenshot、Log 与 Delivery Result。
7. AC Verification Agent 对每条 AC 进行证据匹配和 Verdict。
8. Release Readiness Engine 检查所有审批、Artifact、AC 与 Open Issue。
9. 授权人员完成最终 Release Check 并发布。

4.2 统一局部 Loop 模式

每个阶段统一采用 Produce → Validate/Review → Classify Issue → Route → Targeted Remediation → Re-submit。失败不直接重跑整条 SDLC。

阶段	Produce	Check Gate	失败后的最小返工
Requirement	Use Case Artifact	IT / Business Review	只重新引导相关字段
Content	多渠道多语言内容	Compliance Review	只修改被指出的段落或语言版

			本
Template	Template Configuration	Configuration Validator	重新推荐或修正配置
UAT	Test Run + Evidence	Execution Check	重跑失败 Test Step 或重新取证
AC Verification	AC Verdict	All AC Passed	路由到 Requirement / Content / Template / Test
Release	Release Package	Readiness + Human Check	处理尚未关闭的 Blocker

5. 总体逻辑架构

架构分为体验层、控制面、阶段执行层、共享 AI 能力层、数据与治理层。控制面保持确定性；阶段级 Agent 通过统一 Skill Runtime 与 Artifact Contract 执行。

5.1 体验层

- 统一 Case Workspace：展示阶段进度、表单、Content、Review、Test Evidence 与 Release 状态。
- Case Assistant：用业务语言解释当前阻塞、为什么需要修改、下一步由谁完成。
- 不同 Agent 不直接向用户展示为多个聊天窗口，用户始终面对统一入口。

5.2 控制面

- Workflow Orchestrator：状态机、Gate、Task、Event、Timeout、Retry。
- Case Service：Case ID、当前 Stage、Status、Owner、Blocker。
- Approval Service：IT、Compliance、Business、Release 的审批记录。
- Artifact Version Service：不可覆盖版本、Active Version、Invalidation。
- Issue Service：Issue 生命周期、Owner、Severity、Resolution。

5.3 阶段执行层

- Requirement Intake Agent
- Content Composition Agent
- Template Recommendation & Validation Service
- Test Runner
- AC Verification Agent
- Release Readiness Engine

5.4 共享 AI 能力层

- Skill Registry 与 Prompt Ownership
- Knowledge Service 与权限过滤
- Review Feedback Processing
- Issue Router
- Impact Engine
- Evidence Matching
- Traceability Lookup
- Artifact Diff 与 Change Summary

6. 核心组件详细设计

6.1 SDLC Workflow Orchestrator

Workflow Orchestrator 是总流程的唯一控制器，不依赖 LLM 决定合法状态转换。它消费事件、读取 Case State、检查 Gate，并创建下一任务。

输入	核心处理	输出
Case State、Event、Approval、Open Issue	校验状态转换、Gate、权限、版本依赖	新状态、Stage Task、通知、Audit Event
Issue Routing Result	根据 target_stage 创建返工任务并失效受影响 Artifact	REWORK_REQUIRED 状态和责任人
All Gate Results	判断是否可进入下一 Stage	Stage Transition 或 BLOCKED

允许的核心状态转换

- REQUIREMENT_IN_PROGRESS → REQUIREMENT_REVIEW
- REQUIREMENT_REWORK_REQUIRED → REQUIREMENT_IN_PROGRESS
- CONTENT_DRAFTING → COMPLIANCE_REVIEW
- CONTENT_REWORK_REQUIRED → CONTENT_DRAFTING
- TEMPLATE_READY → UAT_READY
- UAT_COMPLETED → AC_VERIFICATION
- AC_REWORK_REQUIRED → 对应责任 Stage
- ALL_AC_PASSED → RELEASE_CHECK
- RELEASE_APPROVED → RELEASED

6.2 Case Assistant

Case Assistant 是轻量统一交互层，不是主 Agent。它读取 Orchestrator 产生的结构化状态，向用户展示当前阶段、阻塞原因、建议动作和变更影响。它不能批准、跳过 Gate 或直接修改 Case State。

6.3 Requirement Intake Agent

 Agent 延续现有 Phase 1 设计，采用 Goal-driven、Bounded、Code-first Agent Runtime。Goal 是生成完整、一致、经用户确认的 Requirement Artifact 与 AC Package，而不是填满全部 33 个字段。

- 读取 Metadata、Dependency Rules、历史 Use Case 与 MDC Capability Knowledge。
- 每轮只选择 1–3 个最高价值问题。
- 用户回答经 Answer Parsing、Validation、Reducer 更新状态。
- 所有适用 P0 字段确认且无 Conflict/Invalid/Blocker 后，允许最终确认。
- IT Review 退回字段问题时，以 Review Issue 作为新 Event 进入局部返工。

6.4 Content Composition Agent

Content Agent 基于 Confirmed Requirement、AC、历史成功模板和 Content Rules 生成多渠道、多语言内容。它必须支持局部重生成，避免因单一 Compliance Comment 重写全部内容。

能力	说明
Template Retrieval	按 Channel、Language、Business Purpose、Approval Result 检索历史模板
Content Generation	生成 Subject、Body、CTA、Disclaimer 等结构化内容
Localized Variant	生成多语言版本并做跨语言一致性检查

Targeted Regeneration	只修改被审核指出的语言、段落或字段
Compliance Pre-check	提交人工审核前执行规则和历史高频问题检查
Change Summary	生成修改前后 Diff 与重新提交摘要

6.5 AC Verification Agent

AC Verification Agent 读取已版本化的 AC、Test Run、Screenshot、Log、Template 和 Requirement，逐条生成 Evidence-grounded Verdict。

输出字段	说明
verdict	PASS / FAIL / NEED_HUMAN_REVIEW
evidence_refs	Screenshot、Log、Delivery Result、Artifact Version
confidence	自动判断置信度
failure_type	Requirement / Content / Template / Test / Evidence / AC Ambiguity
recommended_route	建议返回阶段，不直接修改 Workflow State
reason	可供和核人理解的解

6.6 Review Feedback Processing

人工审核决定仍由 Reviewer 作出。AI 仅将自然语言评论转换为结构化 Issue，包括问题类型、受影响字段或内容、严重度、证据、是否需要用户补充及推荐返工阶段。该能力作为共享 Skills/Service，不单独建设长期运行的 Review Agent。

6.7 Template Recommendation 与 Validation

Template 阶段优先采用检索、规则和用户确认，而非独立 Agent。推荐逻辑综合 Channel、Language、Message Type、Content Structure、历史通过状态和平台兼容性；Validator 检查 Template 与 Requirement/Content 配置是否一致。

6.8 Test Runner

Test Runner 是确定性执行服务，负责在 UAT 环境触发各 Channel 测试、记录请求与响应、采集 Screenshot、保存 Delivery Status，并产生可被 AC Verification 消费的 Evidence Package。

6.9 Release Readiness Engine

Release 前由规则引擎检查所有必需审批、最新 Artifact、测试结果和 Open Issue。AI 可生成 Release Summary，但不能替代最终 Release Approval。

- Requirement approved
- Content compliance approved
- Template configuration valid
- UAT run completed
- All mandatory AC passed
- No open blocker
- Artifact versions mutually consistent
- Authorized human release approval

7. 各阶段技术设计与局部 Loop

7.1 Requirement Intake 与 IT Review

项目	设计
输入	Raw Requirement、历史 Use Case、Metadata、Capability Knowledge
执行组件	Requirement Intake Agent + Rule Engine
输出	Requirement Draft / Confirmed Requirement / AC Package
检查	Completion Validator + IT/Business Review
失败 Loop	Review Comment → Field Mapping → 只追问受影响字段 → 新 Requirement Version

7.2 Content 与 Compliance Review

项目	设计
输入	Confirmed Requirement、AC、历史模板
执行组件	Content Composition Agent
输出	Content Package（Channel × Language）
检查	Rule Pre-check + Compliance Human Gate
失败 Loop	Comment Parsing → Content Section Mapping → 局部重生成 → 用户确认 → 再提交

7.3 Template Selection / Configuration

项目	设计
输入	Approved Content、Requirement 配置
执行组件	Template Recommendation Skill + Validator
输出	Template Configuration Artifact
检查	兼容性、必填结构、Channel/Language 一致性
失败 Loop	配置 Issue → 重新推荐或修正参数 → 再验证

7.4 UAT Testing

项目	设计
输入	最新 Requirement、Content、Template
执行组件	Test Runner
输出	Test Run、Screenshot、Log、Delivery Result
检查	Execution Gate 与 Evidence Completeness
失败 Loop	环境/脚本/取证问题 → 仅重跑失败步骤

7.5 AC Verification

项目	设计
输入	AC、Test Run、Evidence、所有相关 Artifact Version
执行组件	AC Verification Agent
输出	逐条 Verdict + Evidence + Failure Type
检查	All Mandatory AC Passed
失败 Loop	Issue Router 将失败返回 Requirement / Content / Template / Test / Human

7.6 Release Check

项目	设计
输入	全部审批、Artifact、Test、AC Verdict、Open Issue
执行组件	Release Readiness Engine + Human Approver
输出	Release Package / Release Decision
检查	所有硬 Gate 满足
失败 Loop	生成阻塞清单，路由责任 Owner；不自动 Release

8. Workflow 状态机、事件与 Gate

8.1 建议的 Case Stage

Case Stage 用于描述生命周期位置；Status 用于描述当前阶段中的状态。Stage 与 Status 分离，避免单一枚无限膨。

Stage	典型 Status
REQUIREMENT	IN_PROGRESS / READY_FOR_REVIEW / REWORK_REQUIRED / APPROVED
CONTENT	DRAFTING / USER_CONFIRMATION / COMPLIANCE_REVIEW / REWORK_REQUIRED / APPROVED
TEMPLATE	RECOMMENDING / CONFIGURING / INVALID / READY
UAT	PREPARING / RUNNING / FAILED / COMPLETED
AC_VERIFICATION	VERIFYING / PARTIAL_FAIL / HUMAN_REVIEW / ALL_PASSED
RELEASE	BLOCKED / READY_FOR_CHECK / APPROVED / RELEASED

8.2 关键事件

- REQUIREMENT_SUBMITTED
- REQUIREMENT_CONFIRMED
- IT_REVIEW_APPROVED / CHANGES_REQUIRED
- CONTENT_CONFIRMED

- COMPLIANCE_APPROVED / CHANGES_REQUIRED
- TEMPLATE_CONFIRMED
- UAT_RUN_COMPLETED / FAILED
- AC_VERIFICATION_COMPLETED
- ISSUE_CREATED / RESOLVED
- RELEASE_CHECK_REQUESTED / APPROVED

8.3 Gate 设计

每个 Gate 返回 PASS、FAIL 或 NEED_HUMAN_REVIEW，并包含 reason_code、evidence_refs、required_actions。Gate 本身不得修改业务状态，由 Orchestrator 根据果行。

9. Issue Routing 与 Impact Analysis

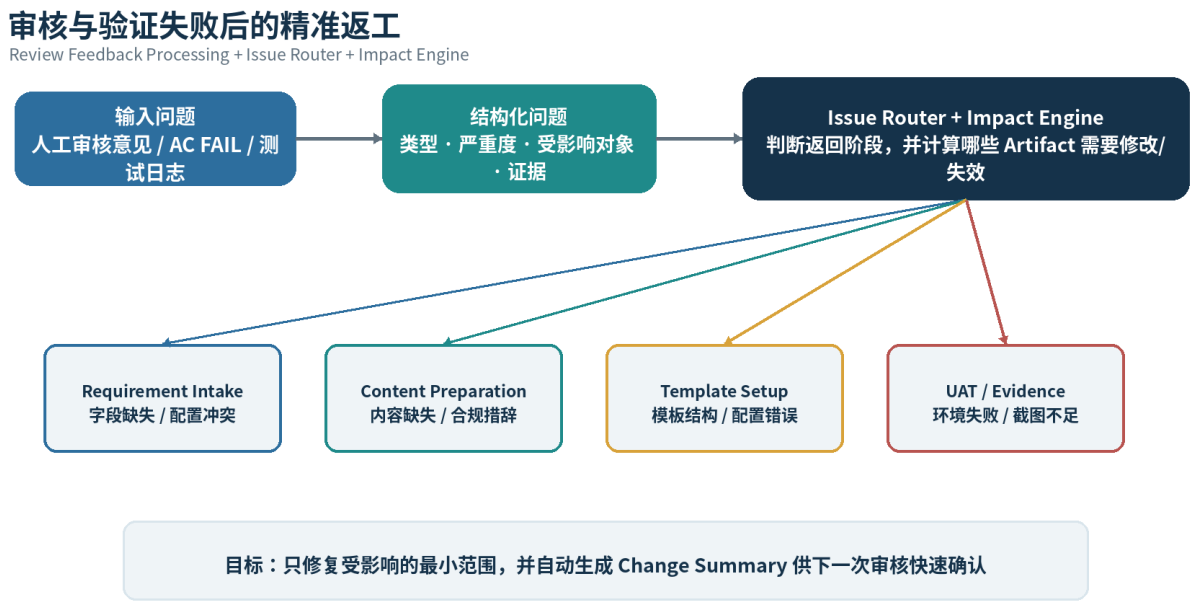


图 3 审核与验证失败后的精准返工

9.1 Issue Taxonomy

Issue Type	典型来源	默认责任阶段
REQUIREMENT_MISSING / CONFLICT	IT Review、AC Fail	Requirement
CONTENT_MISSING / NON_COMPLIANT	Compliance、AC Fail	Content
TEMPLATE_CONFIG_ERROR	Template Validator、UAT	Template
TEST_EXECUTION_ERROR	Test Runner	UAT
EVIDENCE_INSUFFICIENT	AC Verification	UAT/Evidence
AC_AMBIGUOUS	AC Verification	Human / Requirement
UNKNOWN_ROOT_CAUSE	任意阶段	Human Triage

9.2 Issue Router

Issue Router 采用"确定性规则优先 + LLM 分类补充 + 置信度阈值"的方式。高风险或低置信度问题必须人工确认。其输出是 Routing Recommendation，不直接改变 Stage。

9.3 Impact Engine

Impact Engine 依据 Traceability Graph 计算修改会影响哪些 Artifact、审批和测试步骤，并输出 invalidate、regenerate、re-review、retest 清单。

10. Agent、Skill 与 Reducer 运行模型

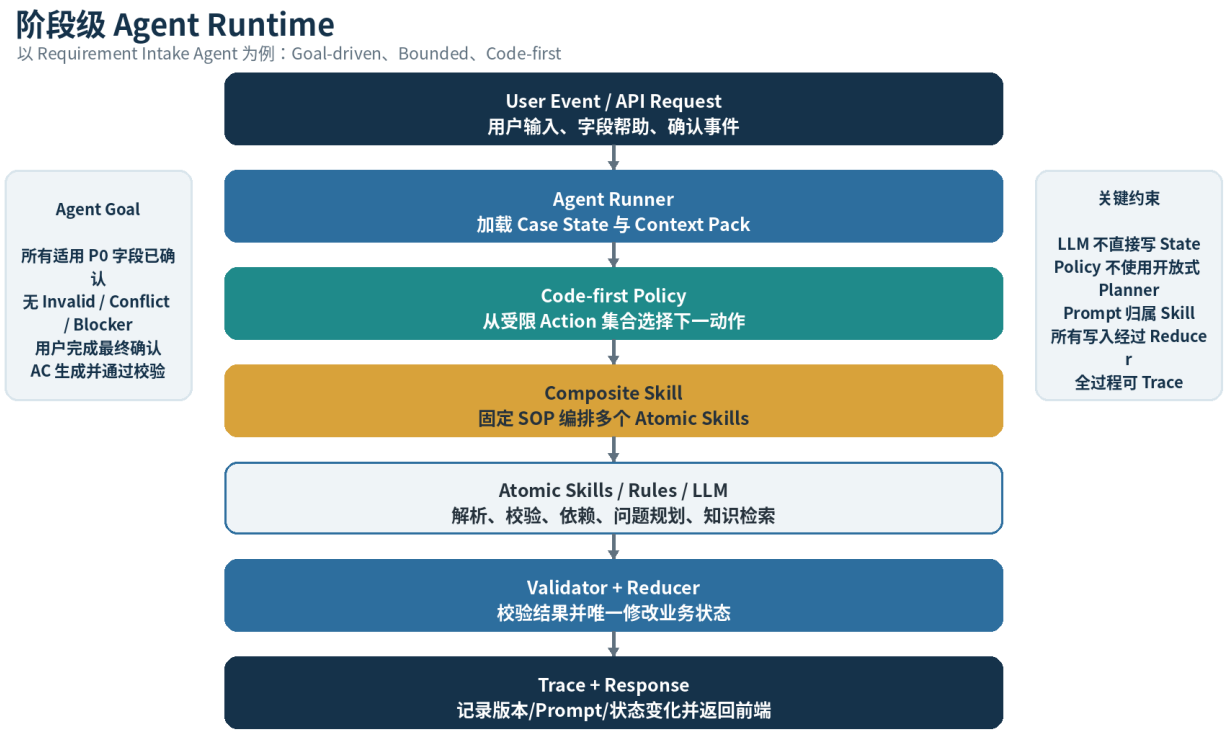


图 4 阶段级 Agent Runtime

10.1 Bounded Agent

阶段级 Agent 只从受限 Action 集合中选择动作；Stage Flow、Gate 和审批不由 LLM 决定。Requirement Intake Agent 的 Action 示例包括 INITIALIZE_INTAKE、PROCESS_USER_ANSWER、PREPARE_NEXT_TURN、PROVIDE_FIELD_HELP、FINALIZE_REQUIREMENT、GENERATE_AND_HANDOFF。

10.2 Composite Skill 与 Atomic Skill

层级	职责	示例
Composite Skill	执行固定业务 SOP，编排多个 Atomic Skills	process_user_answer、prepare_next_turn
Atomic Skill	执行单一可复用能力	answer_parsing、answer_validation、dependency_assessment
Rule / Metadata	提供确定性领域事实，不作为 Skill	dependency_rules、field metadata
Infrastructure	Runner、Registry、Repository、Trace	不作为业务 Skill

10.3 Reducer

Reducer 是 Agent Path 中唯一将"当前 State + 已验证结果"转换为"新 State"的状态更新器。Skill 只提出 Patch，Validator 判断是否合法，Reducer 正式应用并增加 state_version。

- 字段值与状态更新
- Conflict 创建与解决
- Active Question 更新
- Case Stage/Status 更新
- Requirement/AC/Content/Template Artifact 引用更新
- Artifact Invalidation
- Handoff Package 保存
- Audit Event 与 State Version

10.4 Prompt Ownership

每个 LLM Prompt 必须有唯一 owning Skill。Legacy Prompt 文件只能兼容转发，不能保留第二份 Prompt 内容。Skill 记录 skill_version、prompt_version、input_contract、output_contract、validator 和 failure_code。

11. Knowledge 与 Prompt/Skill 治理

11.1 Knowledge 分类

知型	主要用途	推荐存储
Field Knowledge	字段定义、必填条件、示例、Hint	结构化 Metadata
MDC Capability	Channel、Language、Schedule、Fallback 等能力	Capability Catalog + Approved Docs
Policy / Compliance	内容限制、声明、审批规则	版本化文档检索
Historical Success Cases	Use Case、Content、Review、Test、UAT Outcome	结构化索引 + Hybrid Retrieval
Operational / Test Knowledge	错误码、环境限制、失败处理	Runbook + Structured KB

11.2 Knowledge Service

- 结构化查询优先于开放式 RAG。
- 所有回答返回 source、version、section、effective_date。
- 继承来源系统权限，检索前执行 Access Filter。
- Confluence 内容视为数据，不得覆盖 System Instruction。
- 历史案例优先选择最终成功通过 UAT 的版本。

11.3 Skill Governance

- 每个 Skill 具有 Owner、Version、SOP、Prompt、Contract、Validator、Tests 与 Eval Set。
- Skill 发布必须经过回归测试和人工批准。
- Feedback 不得在线自动修改 Prompt 或规则。
- Composite Skill 负责固定业务顺序，Agent 不自由编排全部 Atomic Skills。

12. Case、Artifact、Issue 与 Evidence 数据模型

12.1 Case

字段	说明
case_id	全流程稳定 ID
current_stage / status	当前生命周期位置
active_versions	各 Artifact 当前生效版本
owners	业务、IT、Compliance、Release Owner
open_issue_ids	未关闭问题
pending_tasks	待处理任务
approvals	各 Gate 决策
state_version	并发控制与审计

12.2 Artifact

所有关键产物均采用不可覆盖版本，包括 Requirement、AC、Content、Template Configuration、Review Decision、Test Run、Evidence Package、Verification Report 与 Release Package。

12.3 Issue

字段	说明
issue_id / source_stage	唯一 ID 与来源阶段
issue_type / severity	标准分类与严重度
affected_entities	字段、Content Section、Template、AC、Test Step
evidence_refs	评论、截图、日志等证据
recommended_route	建议责任 Stage/Capability
status	OPEN / IN_PROGRESS / RESOLVED / ESCALATED
resolution	修改内容、版本与审批记录

12.4 Evidence

Evidence 必须绑定 case_id、test_run_id、artifact_versions、channel、language、capture_time、source、hash、access_control 和 retention_policy。

13. Traceability Graph 与版本失效

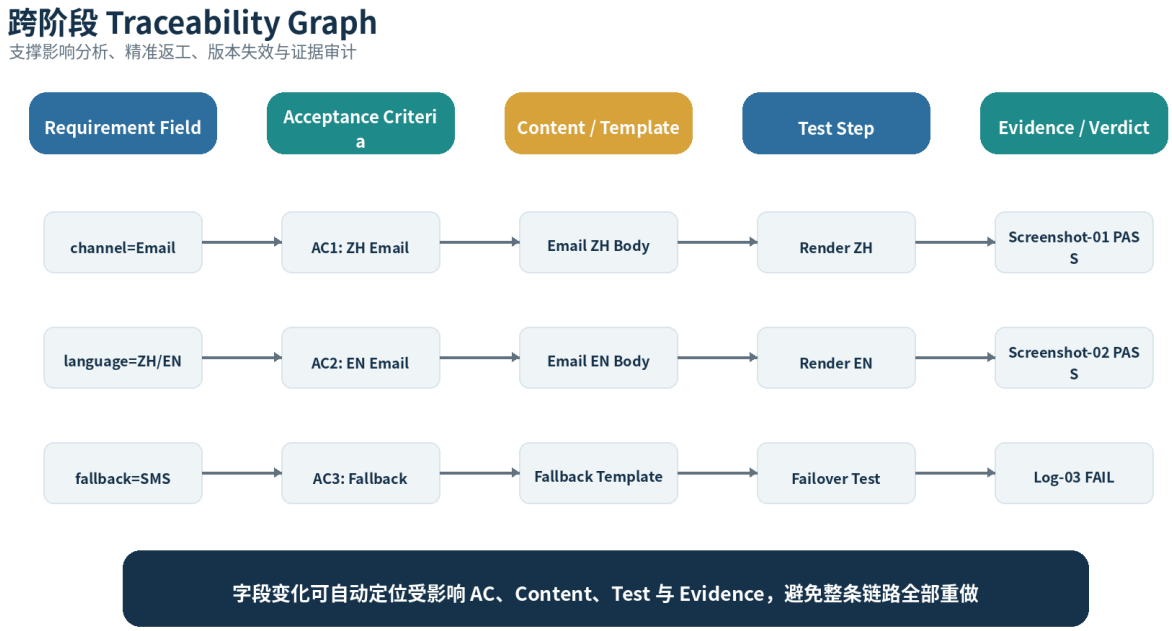


图 5 跨阶段 Traceability Graph

13.1 核心关系

- Requirement Field → generates / constrains → AC
- Requirement Field → constrains → Content / Template
- AC → validated_by → Test Step
- Test Step → produces → Evidence
- Review Issue → affects → Artifact / Field / Section
- Artifact Version → supersedes → Previous Version

13.2 Invalidation Policy

变更	典型影响
Channel / Language 变化	AC、Content、Template、Test Plan、Evidence 失效
Audience 或 Consent 变化	Requirement 需重新审核，可能影响 Content/Compliance
单一 Content 段落变化	对应 Content Version、Compliance Decision、相关 AC Evidence 失效
Template 配置变化	Template Review、受影响 Test Step 与 Evidence 失效
测试环境重跑	Requirement/Content 不失效，仅 Test Run/Evidence 新版本

14. API、事件与接口契约

14.1 API 原则

- 接口围绕 Case、Task、Artifact、Issue、Approval 设计，不围绕聊天 Session 设计。
- Agent 输出必须是结构化 Result，不直接写数据库。
- 所有写请求携带 expected_state_version，支持乐观锁。

- 所有跨服务操作使用 idempotency_key。

14.2 建议核心 API

API	用途
POST /cases	创建 Messaging Case
POST /cases/{id}/events	提交用户或系统事件
GET /cases/{id}	取 Case Workspace 据
POST /cases/{id}/approvals	提交人工审批
POST /cases/{id}/issues	创建结构化 Issue
POST /cases/{id}/tasks/{taskId}/execute	执行阶段任务
GET /cases/{id}/artifacts	读取版本化 Artifact
POST /cases/{id}/uat-runs	触发 UAT
POST /cases/{id}/ac-verification	启动 AC Verification
POST /cases/{id}/release-check	执行 Release Readiness

14.3 Event Contract 示例

Event 至少包含 event_id、case_id、event_type、actor、timestamp、payload、artifact_versions、correlation_id、idempotency_key。

15. 安全、限、人工治理

15.1 Least Privilege

组件	允许操作	禁止操作
Requirement Agent	读写 Requirement Draft Proposal	批准 IT Gate、修改 Compliance Decision
Content Agent	生成 Content Draft	修改 Confirmed Requirement、批准 Compliance
AC Verification Agent	读取 Artifact/Evidence、生成 Verdict	修改 Content 或 Release
Case Assistant	读取状态、解释下一步	改变 Stage、审批、直接写 State
Workflow Orchestrator	执行合法状态转换	绕过 Human Gate

15.2 Human Gates

- 业务最终确认 Requirement
- IT/Business Review Approval
- Compliance Approval
- 必要的 AC Human Review
- Release Approval

15.3 Audit

记录 Actor、Action、Input Artifact Version、Skill/Prompt/Model Version、Tool Call、Validator Result、Reducer Patch、Approval Decision、Evidence、Timestamp 和 Correlation ID。

15.4 Prompt Injection 与数据安全

- 外部文档、历史模板、Reviewer Comment 均作为不可信数据。
- 知识内容不能触发 Tool Call、提升权限或修改 System Instruction。
- 敏感数据在进入模型前执行最小化、Masking 与 Access Control。
- 模型输出必须经过 Schema、Policy 和 Business Rule Validation。

16. 可观测性、评测与 Feedback Improvement Loop

16.1 Trace

每次 Agent/Skill 调用记录 case_id、turn_id、stage、action、composite_skill、atomic_skill、skill_version、prompt_version、model_version、input_artifacts、validation、state_version_before/after、latency、failure_code。

16.2 Stage-level Eval

阶段	核心指标
Requirement	字段预填准确率、错误预填率、无效问题率、平均澄清轮数、IT 一次通过率
Content	Requirement 一致性、局部修改稳定性、用户编辑距离、Compliance 一次通过率
Template	推荐接受率、配置校验准确率、错误配置拦截率
UAT	执行成功率、证据完整率、重跑率
AC Verification	False PASS、False FAIL、Evidence Grounding、Root Cause Accuracy
Overall	Cycle Time、人工触点、返工轮次、自动化率、成本/Case

16.3 Offline Improvement Loop

用户纠正、IT/Compliance 退回、AC Fail 和 Release Blocker 进入 Feedback Store。离线聚类高频问题，映射到具体 Skill/Rule/Knowledge，生成改进候选，经过回归评测和审批后发布。

反示例	改进对象
sender_domain 高频填错	Field Explanation Hint + Answer Validation Rule
同类 Content 多次被 Compliance 修改	Content Prompt + Compliance Pre-check Rule
AC 经常遗漏语言组合	AC Generator + Deterministic Coverage Validator
截图识别 False PASS	Evidence Matching Eval + Confidence Threshold

17. 部署与集成方案

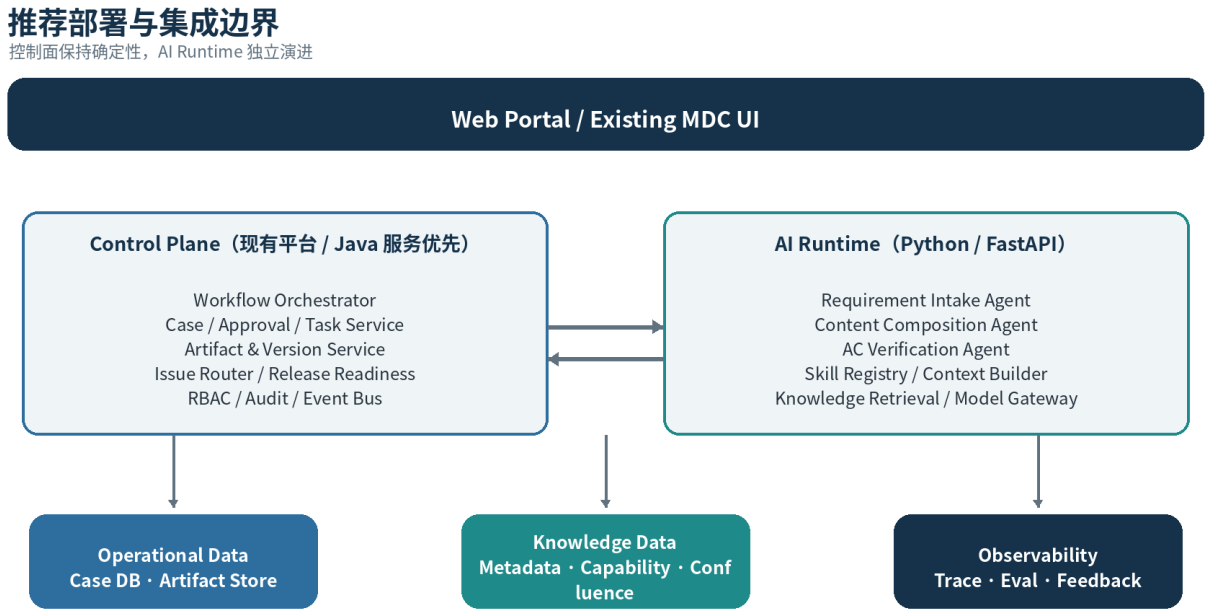


图 6 推荐部署与集成边界

17.1 逻辑部署建议

若现有 MDC 主平台以 Java 为主，建议将 Workflow Orchestrator、Case/Approval/Artifact/Issue 等控制面放在现有 Java 服务体系；AI Runtime 使用 Python/FastAPI 独立部署，通过结构化 API/Event 交互。建可根据有技整。

17.2 集成边界

- AI Runtime 不拥有审批数据库和最终 Stage 转换权。
- Control Plane 不内嵌 Prompt 与模型业务逻辑。
- Artifact 通过 ID/Version 传递，避免发送完整聊天历史。
- 模型调用统一经过 Model Gateway，集中执行限流、审计与安全策略。
- 同步交互用于短任务；UAT、批量生成和 Verification 使用异步 Task/Event。

18. 分阶段实施路线

阶段	建设范围	完成标志
Phase 1 - Requirement	Requirement Intake Agent、Metadata/Rules、AC、Content Handoff	Confirmed Requirement + Validated AC 可供下游消费
Phase 2 - Workflow Foundation	Case State、Stage/Gate、Artifact/Version、Approval、Issue	Requirement 与 Content 纳入同一 Case Backbone
Phase 3 - Content & Compliance Loop	Content Agent、局部重生成、Review Feedback Processing	Content 可生成、审核、精准返工
Phase 4 - Template & UAT	Template Recommendation、Test Runner、Evidence Package	可自动执行 UAT 并生成完整证据
Phase 5 - AC Verification & Routing	AC Verification Agent、Issue Router、	AC Fail 可证据化并精准返回责任阶段

	Impact Engine	
Phase 6 - Release & Improvement	Release Readiness、Feedback Store、Skill Eval/Version	端到端可审计并形成持续改进闭环

18.1 每阶段上线原则

- Feature Flag 控制新旧路径。
- 先 Shadow / Suggestion，再逐步自动执行低风险动作。
- 每阶段具备独立 E2E、回退路径和业务 KPI。
- 不以 Agent 数量作为完成标准，以可验证业务价值作为完成标准。

19. Phase 1 现状映射与后续影响

现有 Requirement Intake Agent 的 Phase 1 改造仍然有效，不需要因总体架构修订而推翻。它将成为 Requirement Stage 内部的阶段级 Agent。

现有能力	在新架构中的位置
initialize_intake / process_user_answer / prepare_next_turn	Requirement Intake Agent Composite Skills
answer_parsing / validation / dependency / completion	Requirement Atomic Skills / Rules
Prompt Ownership / Skill Registry	Shared AI Capability Layer
Reducer-only State Mutation	所有阶段 Agent Runtime 的统一模式
Requirement Artifact / AC / Handoff	跨阶段 Artifact Contract 起点
Feature Flag / Trace / Tests	后[illegible]的工程治理基础

19.1 需要新增但不属于 Phase 1 的基础

- 统一 Case Backbone 与 Stage State Machine
- Approval / Issue / Artifact Version Service
- Review Feedback Processing 与 Issue Router
- Content Agent 与 Template/UAT/Verification 模块
- 跨阶段 Traceability 与 Impact Engine
- Release Readiness 与 Feedback Store

20. 验收标准、非功能要求与风险

20.1 端到端验收标准

- Case 可从 Requirement 创建并经历所有合法 Stage。
- 每个 Stage 有结构化 Input、Output、Gate 与 Rework Path。
- 人工审批不可被 AI 绕过。
- 失败可被分类并路由到正确责任阶段。
- 只失效和重做受影响 Artifact。
- 所有 AC Verdict 可追溯到 Evidence 与 Artifact Version。
- Release 仅在所有硬 Gate 通过后可执行。
- Feedback 可映射到具体 Skill/Rule/Knowledge 改进对象。

20.2 非功能要求

类别	要求
可靠性	所有异步任务幂等；失败可重试；状态转换具备乐观锁
可用性	核心 Portal 与 Control Plane 高可用；AI Runtime 降级时允许人工流程
性能	交互类 AI P95 建　< 10 秒；长任务异步化
□ □	所有关键输入、版本、决策、证据和审批可追溯
安全	RBAC、最小权限、数据脱敏、知识权限继承、Prompt Injection 防护
可测试性	每个 Skill、Gate、Reducer、Router、API 与 E2E 均有测试
可回退性	Feature Flag 与 Legacy/Manual Fallback

20.3 主要风险与缓解

风险	缓解措施
Agent 过度自动化	保持 Workflow-first，审批与 Stage Transition 由代码和人控制
错误返工路由	规则优先、置信度阈值、Human Triage、持续 Eval
知识过期或冲突	Owner、版本、生效日期、来源引用和冲突标记
状态/版本不一致	Reducer-only、Artifact Version、Traceability、幂等与乐观锁
模型输出不稳定	结构化 Contract、Validator、Fallback、离线回归集
范围膨胀	按阶段交付，每阶段明确 Out of Scope 与 KPI

20.4 最终技术定位

目标系统不是“一个主 Agent 管理所有子 Agent”，而是一个由确定性 Workflow Orchestrator 驱动、由阶段级 Agent 和可复用 Skills 增强、由人工 Gate 治理、由局部返工 Loop 和离线 Feedback Loop 持续优化的智能 Messaging SDLC。